Clock Skew Scheduling

Formulations and Algorithms

Wai-Shing Luk

Fudan University

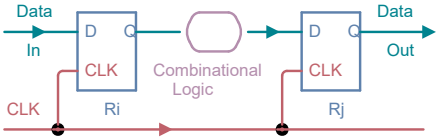
October 21, 2026

Section 1

Introduction

Why clock skew scheduling?

- The clock is **periodic**: only the *relative* arrival times $y_{ij} = u_i - u_j$ matter
- **Useful skew** borrows slack from non-critical paths and lends it to critical ones
- It buys higher frequency, larger margin, or higher **yield**
- Design-flow view: run CTS first, then re-route; **placement is the lever**
- "A railway **timetable**, not a single departure time"



The central idea

- The timing constraints are a **system of difference constraints**

$$u_i - u_j \;\leq\; w_e \qquad \text{for every arc } e = (j, i)$$

- Feasible $\iff$ the **timing constraint graph** (TCG) has **no negative cycle**
- Network form: a **feasible potential problem**

$$\underline{w} \;\leq\; y = Au \;\leq\; \overline{w}$$

- Optimization: a **parametric potential problem**

$$\max\{\, \beta \;:\; y \leq d(\beta),\;\; Au = y \,\}$$

- A negative cycle is not only a certificate — it is the **most critical paths**

Outline

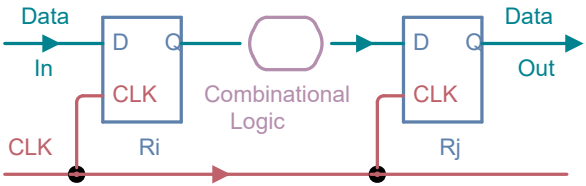


1. Preliminaries: constraints and the timing constraint graph
2. Feasibility, clock-period minimization, parametric methods
3. Multi-parameter problems and the ellipsoid method
4. Delay padding
5. Yield-driven scheduling under process variations
6. Algorithms, clock-tree synthesis, open problems

Section 2

Preliminaries

Local data path



- Register R_i launches data; R_j captures it
- Combinational delay in $[d_{ij}, D_{ij}]$
- Clock arrival times u_i, u_j; skew $y_{ij} = u_i - u_j$
- Only the **difference** matters — the clock is periodic

Setup- and hold-time constraints

- **Setup** (avoids cycle-time violation / zero clocking):

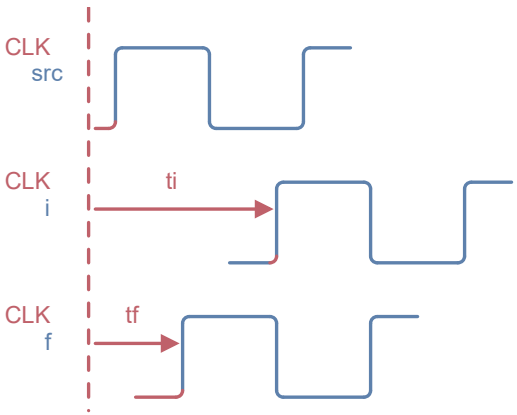
$$y_{ij} \;\leq\; T_{\mathrm{CP}} - D_{ij} - T_{\mathrm{setup}} = \overline{w}_{ij}$$

- **Hold** (avoids a race / double clocking):

$$y_{ij} \;\geq\; T_{\mathrm{hold}} - d_{ij} = \underline{w}_{ij}$$

- **Feasible skew region (FSR):** $\underline{w}_{ij} \leq y_{ij} \leq \overline{w}_{ij}$
- These are exactly **difference constraints** on the arrival times u

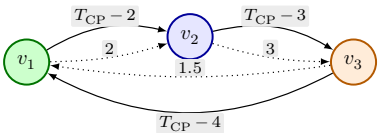
Clock waveforms



- Source clock $\mathrm{CLK}_{\mathrm{src}}$
- Arrivals at i and f delayed by t_i, t_f
- Skew $= t_i - t_f$
- A common shift is **physically meaningless**

The timing constraint graph (TCG)

- One vertex per register; arc weights in the **difference constraints**
- s-**edge** $j \to i$ of weight $\overline{w}_{ij}$ (setup)
- h-**edge** $i \to j$ of weight $d_{ij} - T_{\text{hold}}$ (hold)



Theorem. *The constraints are satisfiable* $\iff$ *the TCG has* ***no negative cycle****.*

- Around any cycle C, $\sum(u_{\text{head}} - u_{\text{tail}}) = 0$, so a cycle is feasible only if its weight is ≥ 0
- A negative cycle certifies that **no schedule exists**

Section 3

Feasibility and Period Minimization

Feasible potential problem

- Find u such that $\underline{w} \leq y \leq \overline{w}, \;\; Au = y$
- **Feasibility condition:** $d^+(P) \geq 0$ for every cycle P
- An infeasible instance returns a **negative cycle**
- **Bellman–Ford** decides one instance in $O(nm)$

Bellman–Ford and lazy evaluation

```
for each v:  u[v] := 0 ;  pi[v] := nil
repeat |V| times:
    for each edge (p,q):
        if u[q] > u[p] + w[p,q]:  u[q] := u[p] + w[p,q] ; pi[q] := p
for each edge (p,q):
    if u[q] > u[p] + w[p,q]:  ->  negative cycle (trace pi)
```

- A negative cycle is detected only at the **end** of a full pass
- **Lazy evaluation**: run the analysis on demand, and **stop at the first negative cycle**
- This turns a global scan into a targeted repair tool

Minimizing the clock period

- Minimize T_{CP} subject to the FSR being satisfiable
- For a cycle C with k setup edges, nonnegativity is

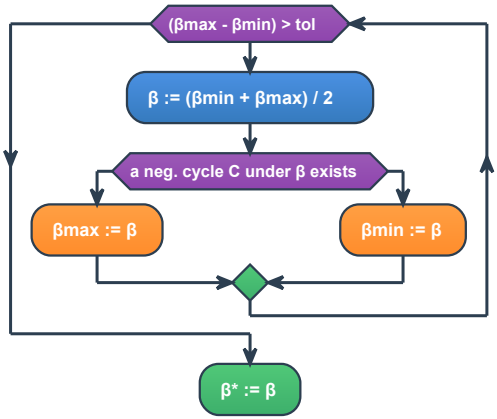
$$T_{\mathrm{CP}} \geq \frac{\sum_{e \in C^{\mathrm{s}}} (D_e + T_{\mathrm{setup}}) - \sum_{e \in C^{\mathrm{h}}} (d_e - T_{\mathrm{hold}})}{k}$$

- Smallest feasible period = a **maximum cycle ratio**
- Constant denominator $\Rightarrow$ the **minimum mean cycle** problem

The parametric potential problem

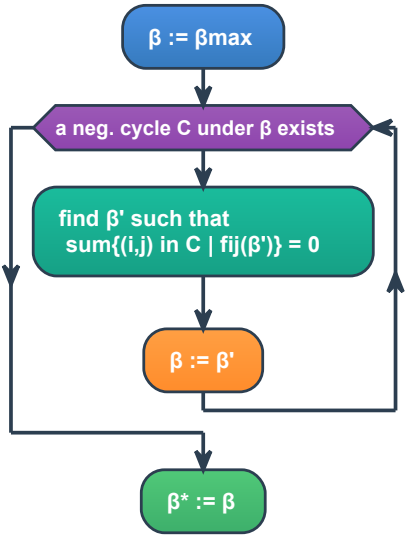
- $$\max\{\, \beta \;:\; y \le d(\beta),\;\; Au = y \,\}, \qquad d(\beta) \text{ monotone decreasing}$$
- Affine $d(\beta) = m - s\beta \Rightarrow$ **minimum cost-to-time ratio**
- Constant $s \Rightarrow$ **minimum mean cycle**
- Monotone g and $f \Rightarrow$ a **unique** solution

Lawler's binary search



- Keep a bracket $[\beta_{\min}, \beta_{\max}]$
- Test the TCG for a negative cycle at the midpoint
- Update a bound and repeat to tolerance
- Simple and robust; one Bellman–Ford pass per test

Howard's policy iteration



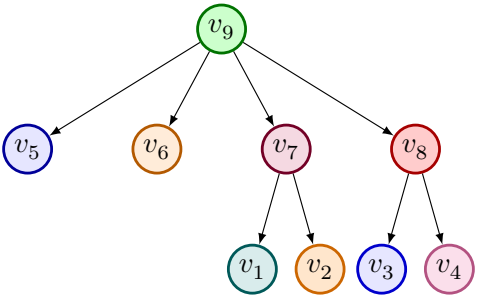
- One chosen outgoing arc per vertex (a **policy**)
- Its cycle yields an improved ratio $\Rightarrow$ a new β
- Recompute shortest paths, then update the policy
- Fast in practice; returns a **criticality hierarchy**
- Hybrid / improved variants interleave bisection pivots

Minimum mean cycle and slack maximization

- **Karp (1978):** closed-form minimum cycle mean in $O(nm)$
- The canonical subroutine for EVEN and slack maximization
- **Slack maximization:** maximize the uniform margin

$$u_i - u_j \;\leq\; \mu_{ij} - \beta$$

- **Minimum balancing:** distribute slack on the critical cycle, contract it to a super-vertex, repeat — the contraction order is the **criticality hierarchy**



- Critical subgraphs form a hierarchy
- Contracting reveals the most restrictive cycles first
- Guides both skew assignment and clock-tree topology

Section 4

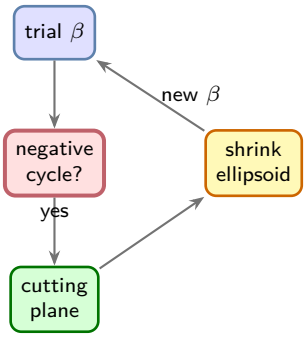
Multi-Parameter Problems

Multi-parameter problems and the ellipsoid method

- Real designs have **several** parameters (period, slack, several yields)

$$\max\{\, g(\beta) : t_i - t_j \le f_{ij}(\beta) \,\}, \quad \beta \in \mathbb{R}^p$$

- A **negative-cycle test is a separation oracle** — one violated cut per step
- **Ellipsoid method:** multi-dimensional bisection; needs only separation
- Scalar bisection $\rightarrow$ **Newton / ellipsoid** when $p > 1$
- Realized by `netoptim`'s `NetworkOracle` + `ellalgo` (companion `digraphx` core; Python · C++ · Rust)



Section 5

Delay Padding

Delay padding

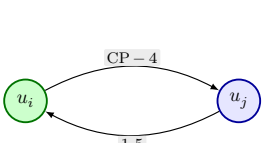
- If the TCG has a negative cycle, **no skew schedule fixes it**
- Change the physical delays: find $p \geq 0$, u with

$$y \leq d + p, \qquad Au = y$$

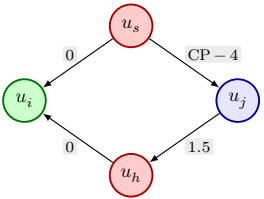
- Minimizing $\sum p$ is the **dual of min-cost flow** — but *impractical*:
  - delay is inserted by **swapping cells**, so $\sum p$ is not the true cost
  - some paths have **no valid position** for insertion

Four structural configurations (PRA)

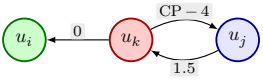
- Decide **where** delay may be inserted *before* deciding *how much*



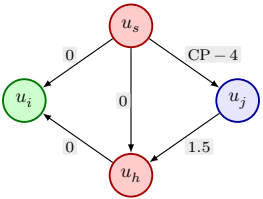
Type (a): none



Type (b): p_s, p_h



Type (c): $p_s = p_h$



Type (d): $p_s \geq p_h$

- The modified TCG makes this a **feasibility problem** again

Discrete padding and its complexity

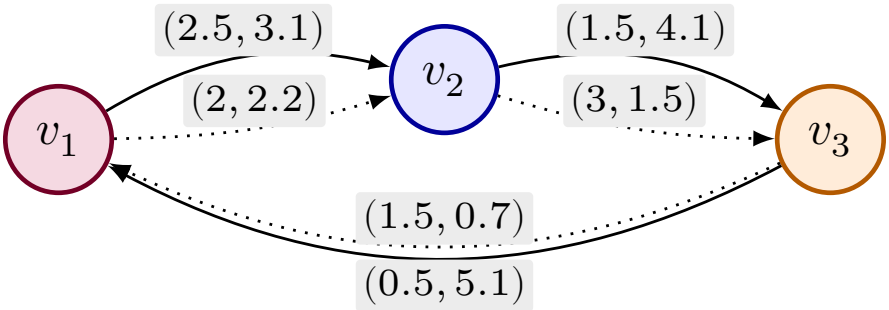
- Real padding is **quantized**: cell substitution or whole buffer stages
- The true feasible set of p is **discrete** (and per-path non-convex)
- The exact problem is a **MILP**, not a network flow
- The continuous relaxation is
  - a **lower bound** on the achievable period, and
  - a guide to *which* paths are worth padding
- "Positions first, then values" keeps the discrete sub-problem tractable

Section 6

Yield-Driven Scheduling

Timing yield

- **Yield** = fraction of chips meeting **all** setup/hold constraints
- Different from a nominal period: we must reason about **distributions**
- Primitive solutions (margin pre-allocation, LCES, incremental slack) all ignore **path-delay variance**



EVEN / PROP / C-PROP

- **EVEN:** equal variances; maximize the uniform margin $\beta \Rightarrow$ minimum mean cycle
- **PROP:** margin proportional to $\sqrt{D_{ij}}$; equalizes margins
- **C-PROP:** maximize β with $u_i - u_j \leq \mu_{ij} - \sigma_{ij}\beta \Rightarrow$ minimum cost-to-time ratio
- C-PROP **reduces to EVEN** when all variances are equal

Problem	$d^{\mathrm{s}}_{ij}(\beta)$	$d^{\mathrm{h}}_{ij}(\beta)$
Min CP	$T_{\mathrm{CP}} - D_{ij} - T_{\mathrm{setup}}$	$d_{ij} - T_{\mathrm{hold}}$
EVEN	$T_{\mathrm{CP}} - D_{ij} - T_{\mathrm{setup}} - \beta$	$d_{ij} - T_{\mathrm{hold}} - \beta$
C-PROP	$T_{\mathrm{CP}} - D_{ij} - T_{\mathrm{setup}} - \sigma_{ij}\beta$	$d_{ij} - T_{\mathrm{hold}} - \sigma_{ij}\beta$

Spatial correlation

- These formulations use per-edge **marginals** and assume **independence**
- Silicon is **correlated**: nearby gates share systematic variation
- Correlation changes both the variance and the effective criticality of a cycle
- It becomes significant at 3 nm and below
- The max-min objective is thus a **surrogate** for the true timing yield

Section 7

Non-Gaussian Delay Models

Why the Gaussian model fails



- The maximum path delay is **asymmetric and heavy-tailed** below 65 nm
- Setup failures depend on the **upper** tail (slow corners)
- Hold failures depend on the **lower** tail (fast corners)
- Gaussian fits the bulk but **misses the tails** — wrong yield

The GEV distribution

- **Generalized Extreme Value:** location μ, scale $\sigma > 0$, shape ξ

$$F(x) = e^{-t(x)}, \qquad t(x) = \left[1 + \xi \, \frac{x - \mu}{\sigma}\right]^{-1/\xi}$$

- Quantile:

$$Q(\beta) = \mu + \frac{\sigma}{\xi}\Big((-\ln \beta)^{-\xi} - 1\Big)$$

- Deterministic equivalents via quantiles, e.g. $T_{\mathrm{CP}} - T_{\mathrm{setup}} - T_{\mathrm{skew}} \geq Q_D(\beta)$
- Monotone $Q \Rightarrow$ a single scalar β; the graph machinery **survives**

Section 8

Algorithms and Clock Trees

Algorithms at a glance

Method	Strategy	Cost / behavior
Bellman–Ford	single feasibility test	$O(nm)$
Karp	minimum mean cycle	$O(nm)$
Lawler	binary search + feasibility test	$O(nm \log(1/\epsilon))$
Howard	policy iteration / cycle cancellation	fast in practice, exact
Young–Tarjan–Orlin	parametric shortest path	$O(nm + n^2 \log n)$

- All share one primitive: **detect and cancel negative cycles**

Reference implementation

- Companion libraries `digraphx` + `netoptim`, sibling packages in **Python, C++, and Rust**
- `digraphx` — combinatorial core: Howard, minimum cost-to-time ratio, parametric solver; **one CSS entry point** for Min CP / EVEN / PROP / C-PROP
- `netoptim` — oracles: `NetworkOracle` (feasibility) and `ClockScheduleOracle` (period $\times$ yield), driven by `ellalgo`
- The oracle returns the **most critical** cycle $\Rightarrow$ a valid, actionable cut
- One algorithm, three languages — useful skew as a **portable** core

From schedule to clock tree

- Realize the arrival times by **DME** + buffer insertion
- Large skews between distant registers are **expensive**: detours, **power**, routing **congestion**
- The schedule is a **delay-only** optimum; a production flow must re-optimize
- **Criticality-ordered topology:** the deepest branch has the smallest variation — let placement follow the tree

Section 9

Closing

Open problems and limitations

- **Open:** physical cost of skew; discrete (MILP) padding; correlation-aware yield; hierarchical / CDC / gated designs; independent evaluation on modern benchmarks
- **Limitations (this survey):** no new experiments
- Four idealizations the framework makes:
  1. **physical realizability** (free continuous arrival times)
  2. **discrete delay** (quantized padding)
  3. **statistical independence** (no spatial correlation)
  4. **flatness** (one synchronous graph)
- The framework is the exact **algorithmic core** of a larger physical problem

Conclusion

- One mathematical core: **difference constraints** + **negative cycle**
- Min period, max slack, max yield all reduce to a **parametric shortest-path** problem
- Fast combinatorial solvers; the critical cycle is directly actionable
- Extends to delay padding, heavy-tailed (GEV) statistics, and multi-parameter scaling via the **ellipsoid method**

Q&A 

Thank you! Questions?